

Expanded Implementation Report

SME Financial Intelligence System Inspired by the MiroFish Workflow

Adapted MiroFish-Style Financial Workflow



Key logic

The LLM does not directly create financial statements. Code parses data, rules validate records, the graph connects evidence, and the LLM explains ambiguity and management questions.

Project purpose	Convert messy SME records into structured, validated, traceable company memory for financial reporting.
Core method	Flexible input -> business ontology -> company knowledge graph -> validation -> statements -> management chat.
Important correction	Do not let the LLM directly generate final accounting numbers. Use rules, graph evidence, and journal logic.
Recommended MVP	Excel/CSV sales, purchases, inventory, chart of accounts, basic income statement, exception dashboard.

Prepared for project planning and system design.

Date: 2026-05-11

Table of Contents

- 1. Executive Summary
- 2. Core Problem and Product Logic
- 3. How the MiroFish Workflow Transfers to SME Finance
- 4. Full Working Sequence
- 5. Data Architecture
- 6. Data Construction Pipeline
- 7. Backend Architecture and Code Services
- 8. API Design
- 9. Frontend Design
- 10. Knowledge Graph Design
- 11. Simulation Replacement: Process Agents
- 12. Memory System: Importance, Risk, Confidence, and Data Quality
- 13. LLM and External Services
- 14. Financial Statement Generation and Audit Trail
- 15. MVP Scope
- 16. Actual Product Roadmap
- 17. Database Structure
- 18. Security, Governance, and Human Review
- 19. Evaluation Metrics and Testing
- 20. Risks and Mitigation
- Appendices: Example Data Journey, Cost Plan, Build Checklist

1. Executive Summary

This report expands the implementation design for an SME Financial Intelligence System inspired by the MiroFish workflow. The system is designed for SMEs that store data in inconsistent formats, including ERP exports, Excel sheets, branch files, invoices, inventory reports, bank statements, and receipts. The main goal is not to ask an LLM to directly create financial statements. The main goal is to build a structured company memory that can support accounting, validation, reporting, and management questions.

Core thesis

The product should be positioned as messy SME data -> structured company memory -> validated financial statements -> management/audit interaction. This is stronger than simply saying AI financial statement generator.

The system uses a staged workflow similar to MiroFish: flexible input, ontology generation, graph construction, downstream reasoning, report generation, and deep interaction. However, the social simulation layer in MiroFish must be replaced by accounting process validation. In finance, correctness matters more than creative simulation.

Dimension	Recommended design
Input	Accept messy SME files, but classify and map them before processing.
Memory	Use raw evidence, structured records, graph relationships, validation results, scores, and audit trail.
Graph	Connect branches, SKUs, suppliers, invoices, payments, inventory movements, accounts, and statement lines.
LLM	Use for ambiguous mapping, explanation, report writing, and chat - not final accounting truth.
Validation	Use deterministic rules for matching, accounting checks, duplicate detection, and missing evidence.
Output	Generate statements with evidence links and exception reports, not just polished text.

2. Core Problem and Product Logic

The core SME problem is not only that files are messy. The deeper problem is that business events are stored separately. Sales, inventory, purchasing, payments, expenses, and accounting records may exist in different files with different names and structures. A financial statement cannot be trusted unless the system can link each number to the business records and logic that created it.

Bad approach	Why it fails
Upload everything to an LLM	The LLM may hallucinate, misread columns, and invent unsupported numbers.
One giant table	It hides relationships between invoices, payments, inventory, accounts, and reports.
Only vector search	Similarity search cannot guarantee accounting logic or traceability.
No human review	High-risk or low-confidence records may enter final statements unchecked.
No audit trail	Users cannot verify where financial statement numbers came from.

The product should instead convert messy records into standardized business objects. Each object must know its source, business process, related entities, validation status, accounting impact, importance score, and evidence trail.

```
Raw SME file
-> parsed rows
```

- > schema-mapped fields
- > structured business records
- > graph relationships
- > validation results
- > accounting journal entries
- > financial statement lines
- > evidence-backed report and chat

3. How the MiroFish Workflow Transfers to SME Finance

MiroFish is useful as an architectural pattern because it uses a staged workflow: ontology generation, graph construction, simulation, report generation, and deep interaction. For SME finance, the same staged idea is useful, but the meaning of each stage must change.

MiroFish concept	Financial AI adaptation	Reason
Ontology generation	Business ontology generation	Identify departments, branches, suppliers, SKUs, invoices, payments, accounts, and report lines.
Graph construction	Company knowledge graph	Connect disconnected SME records into one evidence-backed business memory.
Parallel simulation	Process validation agents	Replace social simulation with accounting and operational checks.
Report generation	Financial statements + management reports	Generate traceable statements, exception reports, and business explanations.
Deep interaction	Management/audit chat	Let users ask why, where, and what evidence supports each number.

Important design boundary

Do not copy MiroFish social simulation directly. A financial system should simulate or validate process flows, not roleplay fake social accounts.

4. Full Working Sequence

The system should run as a pipeline. Each stage produces structured outputs that become the input for the next stage. This is how the code becomes alive.

Adapted MiroFish-Style Financial Workflow



Key logic

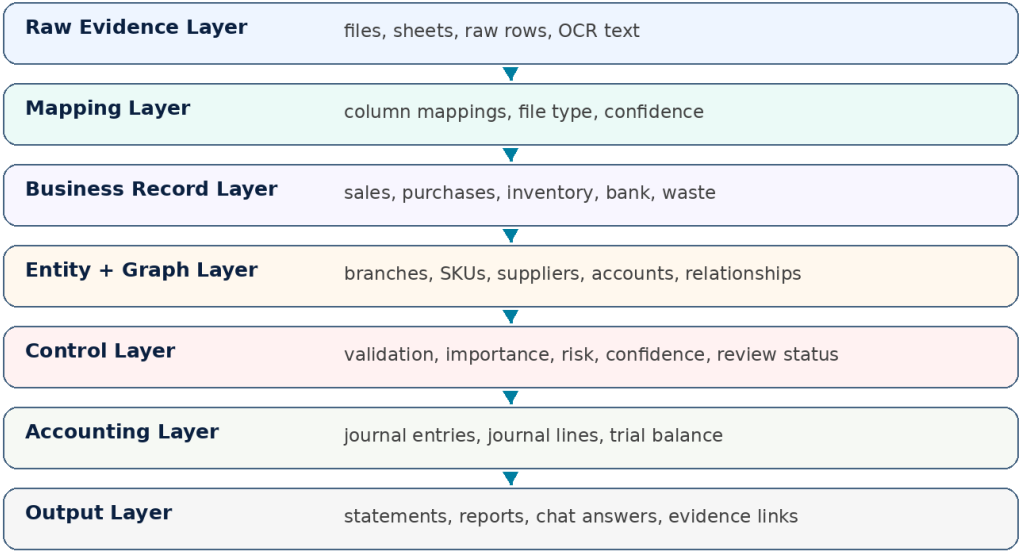
The LLM does not directly create financial statements. Code parses data, rules validate records, the graph connects evidence, and the LLM explains ambiguity and management questions.

1. Create company/project and reporting period.

- 2. Upload raw files: Excel, CSV, ERP export, invoices, receipts, bank statements, inventory records.
- 3. Save raw files and metadata for audit trail.
- 4. Parse file contents into tables, pages, text, or raw row objects.
- 5. Classify file type and business process.
- 6. Map raw columns to standard schema fields.
- 7. Convert rows/documents into structured business records.
- 8. Resolve entities such as supplier names, branch IDs, SKU codes, accounts, and departments.
- 9. Build or apply business ontology.
- 10. Construct company knowledge graph by linking records and entities.
- 11. Run process validation: sales, purchasing, inventory, bank, tax, audit.
- 12. Calculate data quality, confidence, risk, and importance scores.
- 13. Map validated business events to accounting entries.
- 14. Generate journal entries and journal lines.
- 15. Generate statements from journal lines and evidence.
- 16. Generate management report and exception report.
- 17. Enable chat over graph, validation, scores, journals, and statements.

5. Data Architecture

Data Architecture Layers



The data architecture must combine relational storage, flexible JSON, and graph relationships. A purely relational database is good for accounting entries and statements, but not enough for flexible messy input. A graph is good for relationships, but not enough for accounting reliability. JSON is good for flexible raw data, but not enough for controls. The correct design uses all three.

Layer	Main tables/objects	Purpose
Company/period	companies, periods	Define reporting boundary.
Raw evidence	files, file_sheets, raw_records	Preserve original source data and audit trail.
Mapping	column_mappings	Translate messy SME fields into standard business fields.
Business records	business_records	Store standardized rows/documents as business objects.
Entities	entities	Store branches, SKUs, suppliers, accounts, documents, and transactions.
Graph relationships	relationships	Connect business objects across processes.
Control	validation_results, importance_scores	Track correctness, risk, priority, confidence,

		and review need.
Accounting	accounts, journal_entries, journal_lines	Produce accounting numbers from validated data.
Reporting	financial_statements, statement_lines, evidence	Create statements with source traceability.
Interaction	reports, chat_sessions, chat_messages	Store report output and evidence-backed Q&A.

6. Data Construction Pipeline

Data construction is the most important part of the system. The system cannot reason about SME data until raw files have been classified, mapped, normalized, linked, and validated.

6.1 File Parsing

Read Excel, CSV, PDF, image, and ERP exports. MVP should begin with Excel and CSV. OCR should come after the core pipeline works.

6.2 File Classification

Identify whether a file is sales, purchase, supplier invoice, inventory movement, bank statement, chart of accounts, waste record, or expense file.

6.3 Schema Mapping

Map raw columns such as ยอดขาย, Sales, TotalSale, Revenue into a standard field such as sales_amount. This stage needs confidence scoring and human review.

6.4 Record Extraction

Convert each row or document into a typed business object such as SalesTransaction, SupplierInvoice, InventoryMovement, Payment, or WasteRecord.

6.5 Entity Resolution

Merge inconsistent names that refer to the same entity, such as CP Freshmart, C.P. Fresh Mart, and ซีพี เฟรชมาร์ท.

6.6 Relationship Construction

Link records into process chains such as invoice -> PO -> goods receipt -> payment -> AP account.

6.7 Validation and Scoring

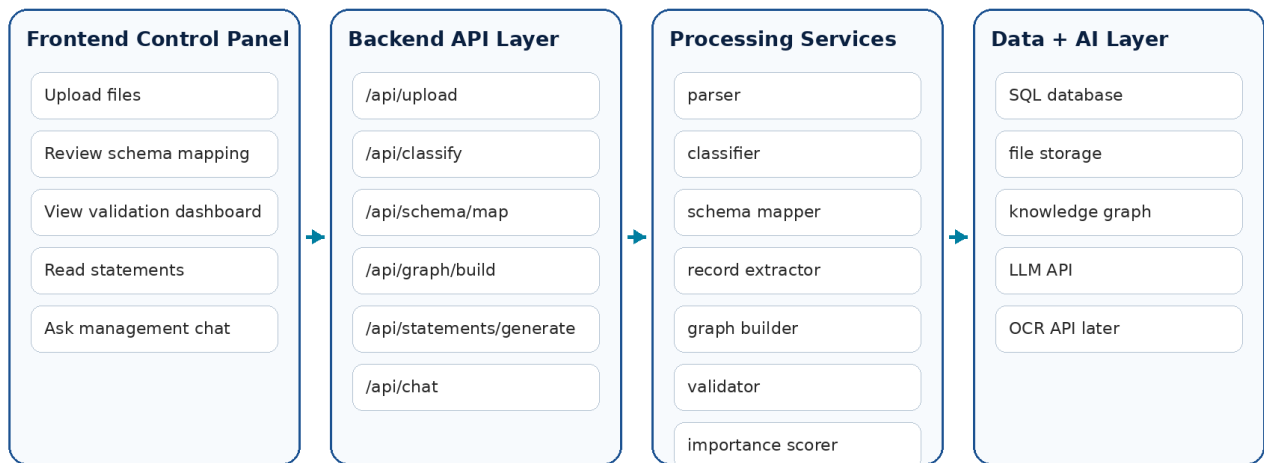
Run deterministic checks and score materiality, risk, exceptions, confidence, and importance.

Hard rule

A number has no meaning until the system knows its context. 50,000 could be sales, cost, payment, inventory value, tax, or discount. Schema mapping and record type classification give numbers meaning.

7. Backend Architecture and Code Services

System Architecture: Frontend, Backend, Services, Data/AI



Design principle: frontend receives decisions; backend controls workflow; services perform jobs; database/graph/LLM store and reason over e

The backend should be modular. Do not put upload, parsing, AI, validation, graph, and reporting into one giant Python file. That would become unmaintainable. Use API files to receive requests and service files to perform specific jobs.

```
backend/  
  main.py           # starts the app and registers APIs  
  config.py         # database, API keys, upload paths, model settings  
  api/              # routes called by frontend  
  services/         # business logic modules  
  agents/           # process-specific validation agents  
  models/           # data classes / schemas  
  storage/          # raw files, parsed records, reports
```

Code service	Main responsibility	LLM needed?
file_parser.py	Read Excel, CSV, PDF, image, ERP export files.	No for Excel/CSV; OCR later.
file_classifier.py	Detect business file type and process.	Sometimes.
schema_mapper.py	Map messy columns to standard fields.	Often useful.
record_extractor.py	Convert rows into structured business records.	No for clean tables.
ontology_service.py	Maintain base ontology and extend for special SME data.	Sometimes.
entity_resolver.py	Merge duplicate supplier, SKU, branch, account names.	Sometimes.
graph_builder.py	Create entities and relationships.	No for rule-based links; LLM optional.
validation_engine.py	Run deterministic checks and exceptions.	No.
importance_scorer.py	Score materiality, risk, exception, criticality.	No, LLM can explain reasons.
accounting_mapper.py	Map business events to journal impacts.	No, should be rule-based.
statement_generator.py	Create trial balance and statements.	No.
report_generator.py	Write readable management report.	Yes for narrative.
chat_agent.py	Answer questions using graph and evidence.	Yes.

8. API Design

API means the backend endpoints that the frontend calls. The API layer should not contain all business logic. It should receive requests, call services, update task states, and return responses.

Endpoint	Purpose	Output
POST /api/upload	Upload raw SME files.	file_id and upload status.
POST /api/classify/{file_id}	Detect file type and process.	file_type, business_process, confidence.
POST /api/schema/map/{file_id}	Suggest standard field mapping.	raw_column -> standard_field mapping.
POST /api/schema/confirm/{file_id}	User confirms or edits mapping.	approved mapping.
POST /api/records/extract/{file_id}	Create business records from raw rows.	record count, record IDs.
POST /api/graph/build	Build company graph.	entity count, relationship count.
POST /api/validation/run	Run process checks.	passed/warning/failed counts.
POST /api/scoring/run	Calculate importance and risk.	scores by record/entity/account/file.
POST /api/statements/generate	Generate financial statements.	statement_id and line items.
POST /api/report/generate	Generate management report.	report_id.
POST /api/chat	Ask evidence-backed questions.	answer + evidence sources.

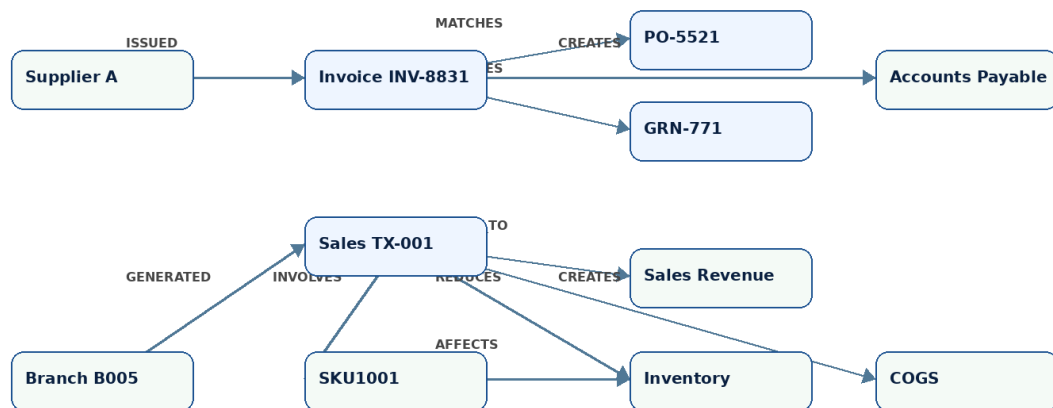
9. Frontend Design

The frontend is the control panel, not the intelligence. It should help users upload files, review mappings, inspect validation issues, read statements, and ask questions. For MVP, Streamlit is enough. For a real product, Vue or React is better.

Page	What it shows	Why it matters
Company setup	Company name, industry, currency, reporting period.	Defines reporting boundary.
Upload page	Raw files and upload status.	Entry point for messy SME data.
Classification page	Detected file type, process, confidence.	User can catch wrong classification early.
Schema mapping review	Raw columns, suggested fields, confidence, approve/edit.	Most important human checkpoint.
Validation dashboard	Passed, warnings, failed, critical exceptions.	Shows data reliability before reporting.
Risk/importance dashboard	High-risk records, missing evidence, material issues.	Prioritizes user attention.
Statement viewer	Income statement, balance sheet, trial balance, evidence.	Shows accounting output.
Management chat	Questions answered from graph and evidence.	Makes the system useful to non-accounting owners.

10. Knowledge Graph Design

Example Company Knowledge Graph



The graph links operational facts to accounting impact and evidence. This is what lets the system explain where statement num

The knowledge graph connects business facts. It is the difference between a normal database of rows and a company memory that understands relationships. The graph should not replace the accounting database; it should sit beside it and link evidence across processes.

Node type	Examples
Operational nodes	Branch, Department, SKU, ProductCategory, Supplier, Customer
Document nodes	Invoice, PurchaseOrder, GoodsReceipt, Receipt, BankStatement, File
Transaction nodes	SalesTransaction, Payment, InventoryMovement, WasteRecord, BankTransaction
Accounting nodes	Account, JournalEntry, JournalLine, StatementLine
Control nodes	ValidationResult, Exception, ReviewTask, ImportanceScore

Edge type	Example
ISSUED_BY	Invoice INV-8831 ISSUED_BY Supplier A
MATCHES	Invoice INV-8831 MATCHES PO-5521
SETTLES	Payment PAY-001 SETTLES Invoice INV-8831
SOLD_AT	Sales TX-001 SOLD_AT Branch B005
INVOLVES	Sales TX-001 INVOLVES SKU1001
POSTS_TO	Sales TX-001 POSTS_TO Sales Revenue Account
AFFECTS	Inventory Movement MOV-001 AFFECTS Inventory Account
SUPPORTED_BY	Statement Line Revenue SUPPORTED_BY Sales Records

11. Simulation Replacement: Process Agents

The original MiroFish simulation concept should be replaced by process agents. These agents do not roleplay departments. They validate business flows. This is the correct financial version of simulation: simulate or check whether the process chain makes sense.

Agent	Input	Main checks	Output
-------	-------	-------------	--------

Sales / Order-to-Cash	POS sales, branch sales, customer payments	Branch exists, SKU exists, revenue mapping, discount anomaly, cash/card split.	Validated revenue, exceptions, journal impact.
Procure-to-Pay	PO, GRN, supplier invoice, payment	3-way match, duplicate invoice, supplier validity, unpaid invoices.	AP validity, unmatched invoices, journal impact.
Inventory-to-COGS	Inventory movements, sales, purchases, waste	Negative stock, abnormal shrinkage, COGS support, waste links.	COGS support, inventory exceptions.
Bank Reconciliation	Bank statements, payments, deposits	Payment/invoice match, sales deposit match, unmatched bank transactions.	Cash validation, reconciling items.
Tax Agent	VAT/tax columns, invoice tax data	Missing tax numbers, abnormal VAT, tax account mapping.	Tax exceptions and tax summary.
Record-to-Report	Journal entries, accounts, statements	Debit-credit balance, account classification, period cutoff.	Trial balance and statements.
Internal Audit	All records, relationships, validation, scores	High-risk exceptions, missing evidence, manual entries.	Audit trail and review queue.

Why this matters

Agents reduce complexity by separating business processes. One LLM cannot safely understand sales, purchasing, inventory, bank, tax, and reporting all at once. Process agents create controlled responsibilities.

12. Memory System: Importance, Risk, Confidence, and Data Quality

The memory system should not store everything equally. It should store raw evidence, structured records, graph links, validation status, scores, and report evidence. Importance scoring helps the system decide what to retrieve first, what to review, and what should appear in reports.

Memory layer	What it stores	Why
Raw source memory	Original files, sheets, raw rows, OCR text.	Audit trail and reprocessing.
Structured record memory	Normalized sales, invoices, payments, inventory.	Standardized processing.
Entity memory	Branches, suppliers, SKUs, accounts, departments.	Business identity and matching.
Relationship memory	Invoice-PO-GRN-payment, sale-SKU-branch-account.	Company knowledge graph.
Validation memory	Passed/warning/failed checks and messages.	Trust and exception control.
Importance/risk memory	Scores, levels, reasons.	Prioritization and retrieval.
Report memory	Statements, line evidence, explanations.	Traceability and management reporting.

12.1 Importance Score Logic

Importance score should be calculated at the small business-object level first, then aggregated upward to files, accounts, departments, branches, and statement lines. Do not score a whole file first. A file is important because of the records inside it.

Importance Score =
 Materiality Score
 + Exception Score
 + Risk Score
 + Account Criticality Score
 + Relationship Score

Example:

Invoice INV-8831

Amount: 1,850,000 THB -> high materiality
Missing goods receipt -> high exception
Affects Inventory and AP -> high account criticality
Total score: 92 / Critical

Score type	Purpose
Field-level score	Detect suspicious values such as missing PO, zero tax, negative quantity, abnormal discount.
Record-level score	Rank invoices, sales transactions, payments, inventory movements, journal entries.
Relationship-level score	Flag broken links such as invoice without GRN, payment without invoice, sales without inventory movement.
Aggregate-level score	Roll up risk to branch, supplier, SKU, account, file, department, and statement line.

12.2 Data Quality Score

Importance is not the same as data quality. A record can be important but low quality. The system should also track data quality based on missing fields, duplicates, invalid dates, unmapped columns, low-confidence OCR, and inconsistent master data.

12.3 Confidence Score

Confidence should be stored for file classification, schema mapping, entity resolution, relationship matching, account mapping, and LLM-generated explanations. Low-confidence outputs should trigger human review.

13. LLM and External Services

The LLM should assist with interpretation and explanation, not replace deterministic accounting logic. It is useful for messy language and flexible formats, but dangerous for final numbers if used without controls.

Use LLM for	Do not use LLM for
Suggest file type when columns are unclear.	Final debit-credit balancing.
Map Thai/English messy columns to standard fields.	Final tax compliance approval.
Extend business ontology for unusual SME data.	Invent missing transactions.
Explain exceptions and management insights.	Approve high-risk records without evidence.
Chat with the user using retrieved graph evidence.	Calculate final statement numbers from raw text alone.

Service/API	When needed	MVP decision
LLM API	Schema mapping, classification, explanations, chat.	Use one cheap reliable provider.
OCR/document extraction API	PDF invoices, receipts, bank statements, scanned documents.	Skip first if Excel/CSV available.
Graph database or memory API	Large company graph, semantic retrieval, temporal memory.	Start with SQL relationships or NetworkX.
Cloud hosting	When users need web access.	Run locally first, deploy after MVP.
Database hosting	For production data and multi-user access.	SQLite first, PostgreSQL later.
File storage	Store original documents and outputs.	Local folder first, cloud storage later.

14. Financial Statement Generation and Audit Trail

Financial statements should be generated from validated business records and journal entries, not directly from LLM text. The LLM can help explain results, but the statement numbers should come from accounting logic.

Sales transaction:

Dr Cash / Accounts Receivable
Cr Sales Revenue
Dr COGS
Cr Inventory

Supplier invoice:

Dr Inventory / Expense
Cr Accounts Payable

Supplier payment:

Dr Accounts Payable
Cr Cash / Bank

Output	Evidence requirement
Trial balance	Every account balance links to journal lines.
Income statement	Revenue, COGS, expenses link to accounts and records.
Balance sheet	Cash, inventory, AR, AP, equity link to journal lines and source records.
Cash flow statement	Later-stage output; requires bank and cash movement mapping.
Exception report	High-risk records, failed validations, missing documents, review tasks.
Management report	Narrative explanation backed by statement lines and graph evidence.

Non-negotiable

Every statement line should answer: where did this number come from, which records support it, which validations were passed, and which exceptions remain open?

15. MVP Scope

The MVP must be narrow. Trying to build full ERP integration, OCR, full tax, balance sheet, cash flow, and a polished Vue frontend at the same time will kill the project. Build the engine first.

MVP component	Include now	Exclude until later
Input	Excel and CSV sales, purchase, inventory, chart of accounts.	Scanned receipts, bank statements, full OCR.
Statements	Revenue, COGS, gross profit, basic trial balance.	Full cash flow and advanced tax reports.
Graph	SQL relationship table or NetworkX.	Paid graph infrastructure unless needed.
Frontend	Streamlit or simple UI.	Full Vue product frontend.
LLM	Schema mapping assistance and explanations.	Autonomous final accounting decisions.
Validation	Basic sales, purchase, inventory checks.	Full audit automation.
Chat	Ask why gross profit changed and show evidence.	General AI assistant for everything.

MVP success question	Expected answer
Can it classify messy Excel/CSV files?	Yes, with confidence and human correction.

Can it map columns to standard fields?	Yes, with mapping review.
Can it create structured records?	Yes, each row becomes a business object.
Can it build relationships?	Yes, branch-SKU-sale, invoice-supplier, inventory-SKU-account.
Can it validate exceptions?	Yes, missing SKU, missing supplier, duplicate invoice, negative stock.
Can it generate basic statements?	Yes, revenue, COGS, gross profit with evidence.
Can it answer management questions?	Yes, using graph and statement evidence.

16. Actual Product Roadmap

Phase	Goal	Main deliverables
Phase 1 Prototype	Prove data pipeline.	Excel/CSV upload, classification, mapping, records, simple graph, basic validation.
Phase 2 MVP	Usable for one SME and one reporting period.	Mapping review UI, exception dashboard, basic statements, evidence links, simple chat.
Phase 3 Pilot	Test with real SME data.	OCR invoices, bank import, entity resolution, branch/SKU analysis, accountant comparison.
Phase 4 Product	Prepare for paying users.	PostgreSQL, cloud deployment, auth, permissions, backup, production frontend, audit logs.
Phase 5 Scale	Multi-company and integrations.	ERP connectors, scheduled ingestion, advanced reporting, compliance packs, API integrations.

17. Database Structure

The database should not be one table called data. It needs clear layers that preserve raw evidence, standardize business records, connect graph relationships, store validation, create accounting entries, and trace statement lines back to evidence.

Table	Key fields	Purpose
companies	company_id, company_name, industry, currency	Company/project boundary.
periods	period_id, company_id, start_date, end_date, status	Reporting period boundary.
files	file_id, filename, path, file_type, confidence	Uploaded file metadata.
file_sheets	sheet_id, file_id, sheet_name, row_count	Excel sheet tracking.
column_mappings	file_id, raw_column, standard_field, confidence	Messy column to standard schema mapping.
raw_records	raw_record_id, file_id, row_number, raw_data_json	Original row-level evidence.
business_records	record_id, record_type, amount, normalized_data_json	Standardized business objects.
entities	entity_id, entity_type, entity_name, attributes_json	Graph nodes and master data.
relationships	source_entity_id, relationship_type, target_entity_id, evidence	Graph edges.
validation_results	record_id, validation_type, status, severity, message	Control checks and exceptions.
importance_scores	object_type, object_id, total_score, level, reason	Risk/materiality priority.
accounts	account_code, account_name, type, statement_type	Chart of accounts.
journal_entries	entry_id, period_id, source_record_id, status	Journal header.
journal_lines	entry_id, account_id, debit, credit	Debit/credit lines.
financial_statements	statement_id, period_id, statement_type	Statement header.
statement_lines	statement_id, line_name, amount, account_id	Statement numbers.
statement_line_evidence	statement_line_id, record_id, journal_entry_id, file_id	Traceability of each statement line.
reports	report_id, period_id, report_markdown/json	Management report.
chat_messages	chat_session_id, role, message_text, evidence_json	Evidence-backed chat history.

18. Security, Governance, and Human Review

Financial data is sensitive. A reasonable product must include controls beyond AI logic.

Area	Required design
Human-in-the-loop	Review low-confidence mappings, critical exceptions, unclear account classification, tax-sensitive records.
Audit trail	Link every statement number to source file, row, record, validation, journal, and evidence.
Permissions	Separate owner, accountant, auditor, branch user, and admin roles.
API key security	Store LLM/OCR keys in environment variables, not code. Never expose keys in frontend.
Data privacy	Encrypt sensitive files, separate company data, keep access logs.
Versioning	Track uploaded file versions, mapping versions, validation rules, statement generations, corrections.
Backup and recovery	Regular database and file backups. Ability to restore previous statement versions.
Error handling	Unreadable files, missing columns, LLM timeout, OCR failure, duplicate files, invalid mappings.

What was missing before

The biggest missing pieces were human review, master data management, data quality scoring, confidence scoring, audit trail, versioning, security, and evaluation metrics. Without these, the idea sounds impressive but not operationally credible.

19. Evaluation Metrics and Testing

The project needs metrics. Without metrics, it is just a story. Evaluation should compare system output against accountant-prepared outputs and known source records.

Metric	What it measures	Target for MVP
File classification accuracy	Correctly identifies sales, purchase, inventory, bank, COA files.	>85% after human correction loop.
Schema mapping accuracy	Correct raw column to standard field mapping.	>90% on tested templates.
Record extraction accuracy	Rows correctly converted to business objects.	>95% for Excel/CSV.
Validation precision	Exceptions flagged are real issues.	High precision over recall at first.
Statement variance	Difference from accountant-prepared statement.	Explainable and decreasing over iterations.
Evidence coverage	Percentage of statement lines with source evidence.	100% for MVP statement lines.
Human review reduction	Time saved for mapping/checking.	Measure manually in pilot.
LLM cost per file	Average API cost per processed file.	Low enough for SME pricing.
Processing time	Time from upload to report.	Acceptable for batch finance workflow.

20. Risks and Mitigation

Risk	Why it is serious	Mitigation
Wrong schema mapping	All downstream numbers become wrong.	Confidence score, preview rows, human approval, templates.
LLM hallucination	System may produce unsupported explanations.	Only use retrieved evidence; require source references.
Weak master data	Duplicate suppliers/SKUs fragment the graph.	Entity resolution and manual merge workflow.

Missing source records	Statements may be incomplete.	Data completeness checks and exception report.
Overbuilding	Project becomes too big and never works.	Start with Excel/CSV, income statement, limited processes.
Accounting rule errors	Statements become unreliable.	Rule library, accountant review, test cases.
Security failure	Financial data leak.	Access control, encryption, audit logs, secure API keys.
No evidence trail	Users cannot trust outputs.	Statement-line evidence table and raw record links.

Appendix A: Example Data Journey

This example shows how one raw sales row becomes financial statement evidence.

Raw row in Excel:

วันที่ = 2026-03-01

สาขา = B005

รหัสสินค้า = SKU1001

จำนวนขาย = 12

ยอดขาย = 960

ต้นทุน = 720

Schema mapping:

วันที่ -> date

สาขา -> branch_id

รหัสสินค้า -> sku_id

จำนวนขาย -> quantity_sold

ยอดขาย -> sales_amount

ต้นทุน -> cost_amount

Business record:

SalesTransaction SALE-001

branch_id = B005

sku_id = SKU1001

sales_amount = 960

cost_amount = 720

Graph relationships:

SALE-001 SOLD_AT Branch B005

SALE-001 INVOLVES SKU1001

SALE-001 POSTS_TO Sales Revenue

SALE-001 CREATES COGS

SALE-001 REDUCES Inventory

Journal impact:

Dr Cash / AR 960

Cr Sales Revenue 960

Dr COGS 720

Cr Inventory 720

Statement impact:

Revenue +960

COGS +720

Gross Profit +240

Evidence:

Statement line links back to SALE-001 and the original Excel row.

Appendix B: Rough Cost Plan

The code files themselves are free if built by the developer. Cost comes from external services. The first version should be cheap. Do not buy every service at once.

Stage	Recommended services	Estimated monthly cost
Student prototype	Local laptop, SQLite, pandas, Streamlit, one low-cost LLM API.	\$5-\$30
Working MVP	Small backend hosting, LLM API, local/SQLite database, no heavy OCR.	\$30-\$150
Pilot	OCR for invoices, managed database, cloud storage, more LLM usage.	\$150-\$600
Production-ish	PostgreSQL, graph DB/Zep/Neo4j, cloud backend, monitoring, security, OCR at scale.	\$600-\$3,000+

Cost control principle

Use code to parse and summarize files before sending data to the LLM. Sending full ERP files directly to an LLM is expensive and unreliable.

Appendix C: Build Checklist

Build item	Status target for MVP
Upload Excel/CSV	User can upload and system saves file metadata.
File classifier	System detects sales/purchase/inventory/COA files.
Schema mapper	System suggests mappings with confidence and user approval.
Record extractor	Rows become standardized business records.
Entity resolver	Branches, SKUs, suppliers, accounts are identified.
Graph builder	Records connect to entities and accounts.
Validation engine	Basic exceptions detected.
Importance scorer	High-risk/material records ranked.
Accounting mapper	Business events create journal impacts.
Statement generator	Revenue, COGS, gross profit generated.
Evidence links	Statement lines link to records and files.
Chat agent	Can answer simple why questions with evidence.

Final Conclusion

The idea is feasible and meaningful if implemented as a structured financial data pipeline. The MiroFish workflow helps because it shows how flexible raw input can become ontology, graph memory, report generation, and deep interaction. But for finance, the system must be stricter than MiroFish. It must use validation rules, accounting mapping, audit trail, confidence scoring, importance scoring, and human review.

Best final positioning

An SME financial intelligence platform that accepts flexible raw business data, maps it into a business ontology, constructs a company knowledge graph, validates records through process-specific controls, scores risk and importance, generates traceable financial statements, and supports management/audit interaction through evidence-backed chat.